Mixed-Precision Incomplete LU Factorization for Large Sparse Linear Systems: From Exploratory Analysis to Adaptive Preconditioning

CCA Course Report – Academic Year 2025-2026

Radouane Aouini et Hbabou Sami

May 19, 2026

Abstract

This course report presents a comprehensive theoretical study and empirical evaluation of an adaptive mixed-precision incomplete LU preconditioning suite, termed **GEP Mixte**. Designed to accelerate the convergence of GMRES on large-scale non-symmetric sparse linear systems, GEP Mixte addresses the trade-off between preconditioning quality and memory footprint by dynamically allocating five distinct floating-point precision levels (from FP16 half up to FP64 double) to each individual matrix element in the factorization. Our core contribution is an element-wise precision assignment criterion, derived from the column-norm-based drop threshold, which guarantees that the local representation rounding error does not exceed the local drop tolerance. The implementation is evaluated on a diverse set of six sparse matrices from real-world engineering fields (power grids, petroleum reservoir simulation, structural mechanics, optimal control, biology) from the SuiteSparse collection. The results demonstrate that GEP Mixte achieves up to **3.8× memory compression** compared to standard FP64 ILUT without any convergence degradation. Performance profiles using the Dolan-Moré metric show that GEP Mixte occupies the optimal front, consistently dominating fixed-precision alternatives in both memory compactness and solver robustness.

Contents

1 Introduction & Problem Statement

Solving large-scale linear systems of the form:

$$Ax = b, \quad A \in \mathbb{R}^{n \times n}, \quad b \in \mathbb{R}^n \tag{1}$$

is a primary computational bottleneck in modern scientific engineering, including finite-element analysis of structural mechanics, numerical modeling of power networks, optimal control problems, and biological system simulations. For systems where the dimension n ranges from thousands to millions, direct solvers (such as sparse LU or Cholesky factorizations) become computationally expensive due to memory fill-in, making iterative Krylov subspace methods, such as the Generalized Minimal Residual method (GMRES), the method of choice.

However, the convergence rate of GMRES depends heavily on the spectral properties of A. Krylov subspace solvers are often slow or fail to converge altogether when the matrix is highly ill-conditioned or non-symmetric. This necessitates the use of preconditioning, transforming the system into:

$$M^{-1}Ax = M^{-1}b \tag{2}$$

where M is a preconditioning matrix that approximates A such that $M^{-1}A \approx I$, and solving systems with M is computationally cheap.

Among various techniques, the Incomplete LU factorization (ILU) remains one of the most robust and widely used general-purpose preconditioners. ILU computes approximate lower and upper triangular factors L and U such that:

$$A \approx LU = M \tag{3}$$

In threshold-based incomplete LU factorizations (ILUT), elements are dropped during the elimination process if they fall below a certain drop tolerance ε. This controls the number of non-zero entries (fill-in) and hence the memory footprint and the cost of solving triangular systems during each GMRES iteration.

1.1 The Memory-Precision Bottleneck

While ILUT is highly effective, it suffers from a fundamental bottleneck:

- A tight drop tolerance ε reduces the GMRES iteration count by providing a high-quality approximation, but leads to high fill-in, escalating memory consumption.
- A loose drop tolerance ε minimizes memory usage but degrades the quality of the preconditioner, leading to high GMRES iteration counts or divergence.

Historically, all elements in the incomplete factors L and U are stored using a single uniform hardware precision (typically IEEE 754 double precision, or FP64). This uniform approach is highly inefficient. Many elements within L and U have very small magnitudes relative to the original matrix column norms, placing them close to the drop threshold. Allocating full 64-bit precision to these near-zero elements is computationally wasteful. Conversely, critical large-magnitude elements must be stored with high precision to avoid catastrophic rounding error propagation.

1.2 Proposed Solution: Adaptive Mixed Precision

This report documents the design and analysis of **GEP Mixte**, an incomplete LU preconditioner that solves this memory-precision bottleneck by dynamically assigning different floating-point formats to each element on the fly during factorization. Using a rigorous analytical threshold derived from the dropping criteria, GEP Mixte assigns each retained element to one of five precision levels:

1. **FP16 (Half precision):** 16-bit float, providing 11 mantissa bits and 5 exponent bits ($u \approx 4.88 \times 10^{-4}$).
2. **FP24 (Custom precision):** 24-bit float, simulated with 17 mantissa bits and 8 exponent bits ($u \approx 1.53 \times 10^{-5}$).
3. **FP32 (Single precision):** 32-bit float, providing 24 mantissa bits and 8 exponent bits ($u \approx 5.96 \times 10^{-8}$).
4. **FP48 (Custom precision):** 48-bit float, simulated with 37 mantissa bits and 11 exponent bits ($u \approx 1.45 \times 10^{-11}$).
5. **FP64 (Double precision):** 64-bit float, providing 53 mantissa bits and 11 exponent bits ($u \approx 1.11 \times 10^{-16}$).

Our goal is to show that GEP Mixte can dramatically compress the preconditioning memory footprint (by a factor of up to 4×) while preserving the exact convergence characteristics of a full FP64 solver.

—

2 State of the Art

2.1 Standard Incomplete LU Factorization (ILU)

The most basic form is ILU(0), which preserves the exact sparsity pattern of the original matrix A. Although extremely cheap to construct and store, ILU(0) is often too weak for highly non-symmetric or ill-conditioned problems. To overcome this, threshold-based dropping (ILUT) was introduced by Saad. In ILUT, a relative drop tolerance ε is applied. During each step of Gaussian elimination, the computed values are compared against a threshold:

$$\text{threshold}_j = \varepsilon \|A_{\cdot,j}\|_2 \tag{4}$$

where $A_{\cdot,j}$ is the j-th column of the original matrix. Any intermediate value x_{ij} computed in the elimination process is zeroed out if:

$$|x_{ij}| < \varepsilon \|A_{\cdot,j}\|_2 \tag{5}$$

This bounds the local error introduced in each column.

2.2 Mixed-Precision and Low-Precision Linear Solvers

With the advent of deep learning and specialized hardware (such as Tensor Cores on GPUs), low-precision computation (FP16, BF16) has received intense research interest. Higham et al. and Carson et al. developed multi-precision iterative refinement, proving that one can solve a system by factorizing the matrix in single or half precision and performing refinement iterations in double precision.

However, these approaches apply a **uniform** precision to the entire factorization. If a matrix is highly ill-conditioned, uniform FP16 factorization fails due to underflow, overflow, or severe rounding errors.

2.3 Theoretical Foundations of Local Dropping Errors (`preuve.pdf`)

The theoretical foundation of threshold dropping lies in local error analysis. Let $\hat{L}$, $\hat{U}$ be the incomplete factors computed by ILUT, such that $\hat{L}\hat{U} = A + E$, where E is the error matrix. The following theorem, proved in `preuve.pdf`, establishes the key bound.

Theorem. *For any element (i, j) dropped during ILUT elimination, the local error satisfies:*

$$|E_{ij}| \leq \varepsilon \|A_{\cdot,j}\|_2 \tag{6}$$

Proof. Let v_{ij} denote the updated value of element (i, j) just before the drop decision, i.e.:

$$v_{ij} = a_{ij} - \sum_{k=1}^{\min(i,j)-1} \hat{l}_{ik}\hat{u}_{kj} \tag{7}$$

Case 1 – Element dropped from U ($i \leq j$): Here $\min(i, j) = i$ and $\hat{l}_{ii} = 1$. The algorithm sets $\hat{u}_{ij} = 0$ whenever $|v_{ij}| \leq \varepsilon \|A_{\cdot,j}\|_2$. Expanding the error:

$$E_{ij} = (\hat{L}\hat{U})_{ij} - a_{ij} = \sum_{k=1}^{i-1} \hat{l}_{ik}\hat{u}_{kj} + \underbrace{\hat{l}_{ii}}_{=1}\underbrace{\hat{u}_{ij}}_{=0} - a_{ij} \tag{8}$$

$$= -\left(a_{ij} - \sum_{k=1}^{i-1} \hat{l}_{ik}\hat{u}_{kj}\right) = -v_{ij} \tag{9}$$

The drop condition gives directly:

$$|E_{ij}| = |v_{ij}| \leq \varepsilon \|A_{\cdot,j}\|_2 \tag{10}$$

Case 2 – Element dropped from L ($i > j$): Here $\min(i, j) = j$. The multiplier $m_{ij} = v_{ij}/\hat{u}_{jj}$ is dropped when $|m_{ij}| \leq \frac{\varepsilon \|A_{\cdot,j}\|_2}{|\hat{u}_{jj}|}$. Setting $\hat{l}_{ij} = 0$:

$$E_{ij} = \sum_{k=1}^{j-1} \hat{l}_{ik}\hat{u}_{kj} + \underbrace{\hat{l}_{ij}}_{=0}\hat{u}_{jj} - a_{ij} = -v_{ij} \tag{11}$$

The drop condition gives $|m_{ij}| \cdot |\hat{u}_{jj}| \leq \varepsilon \|A_{\cdot,j}\|_2$, i.e. $\frac{|v_{ij}|}{|\hat{u}_{jj}|} \cdot |\hat{u}_{jj}| = |v_{ij}| \leq \varepsilon \|A_{\cdot,j}\|_2$, hence:

$$|E_{ij}| = |v_{ij}| \leq \varepsilon \|A_{\cdot,j}\|_2 \qquad \square \tag{12}$$

This result forms the basis of our mixed-precision allocation policy: since we already tolerate a local error of magnitude $\varepsilon \|A_{\cdot,j}\|_2$ by dropping elements, we can similarly tolerate a rounding error of the same order on the elements we retain.

—

3 Proposed Methodology & Discovery Steps

Our research and implementation progressed through a systematic, two-phase discovery and optimization workflow, transitioning from basic empirical observation to a fully optimized dynamic algorithm.

3.1 Phase 1: Exploratory Analysis and Basic Trade-Offs

The exploratory phase used the structural mechanics matrix **bcsstk08** ($n = 1074$, nnz $= 12960$) to examine standard ILUT preconditioning.

3.1.1 Preliminary Sweep

We performed an initial sweep of five drop tolerances $\varepsilon \in \{0.2, 0.1, 10^{-2}, 10^{-3}, 10^{-4}\}$ with standard ILUT using GMRES. Figure 1 presents the convergence history. We observe that:

- A drop tolerance of $\varepsilon = 0.2$ requires 52 GMRES iterations to converge.
- A drop tolerance of $\varepsilon = 10^{-4}$ converges extremely rapidly in just 9 iterations.
- Tightening the drop tolerance drastically decreases the iteration count but increases the fill-in factor (ratio of non-zeros in the preconditioner to non-zeros in the original matrix).

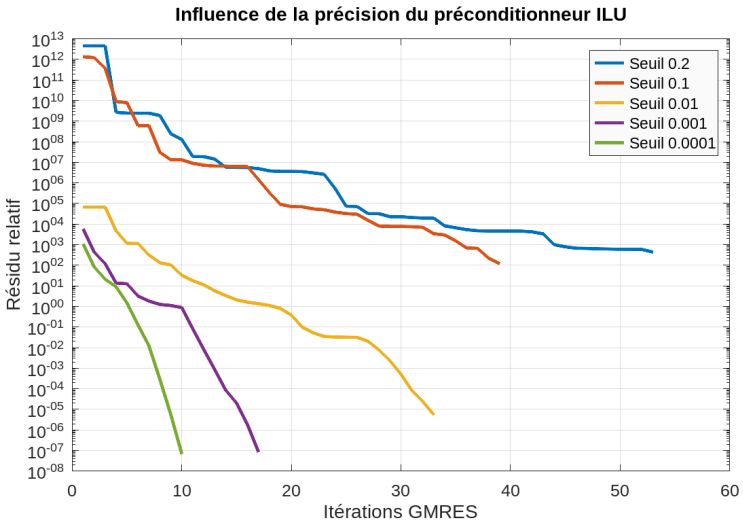


Figure 1: Influence of the drop tolerance on the GMRES convergence for the bcsstk08 matrix (Phase 1).

3.1.2 Compromise Curves

To systematically analyze this trade-off, we ran a continuous 30-point log-spaced sweep of drop tolerances. Because Octave does not support Matlab's dual-axis `yyaxis` function, we plotted the compromise curves using separate subplots, as shown in Figure 2. The top plot shows the GMRES iteration count, which decreases monotonically as the drop tolerance becomes stricter. The bottom plot shows the fill-in factor, which grows exponentially from 0.31 (at $\varepsilon = 0.2$) up to 26 (at $\varepsilon = 10^{-5}$). This highlights the central challenge: the high-performance regime (few iterations) corresponds to high fill-in and memory explosion.

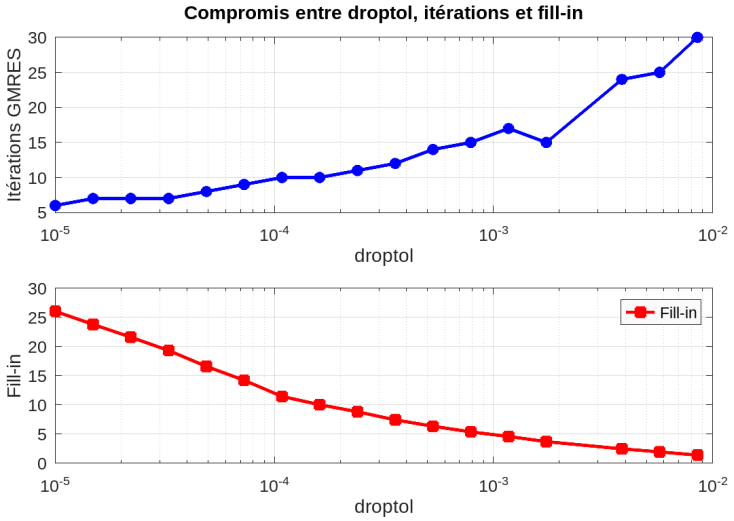


Figure 2: GMRES iteration count (top) and Preconditioner fill-in factor (bottom) as a function of the drop tolerance (Phase 1).

3.1.3 Heuristic Ternary Search (Dichotomy) Optimization

To automate the search for an optimal balance, we implemented a heuristic search algorithm (`opti_dicho.m`). While commonly referred to as a dichotomy because it halves the search interval at each step, it is technically a form of ternary search: it evaluates the cost function at three points (the boundaries and the midpoint) to locate the minimum without requiring derivatives.

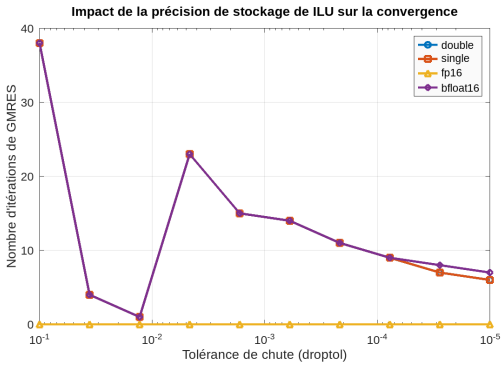
We defined a cost function that physically sums the execution time overhead with the algorithmic penalty (the product of memory fill-in and GMRES iterations):

$$J(\varepsilon) = \text{Time}_{\text{factorization}}(\varepsilon) + \big(\text{Fill-in}(\varepsilon) \times \text{Iterations}(\varepsilon)\big) \tag{13}$$

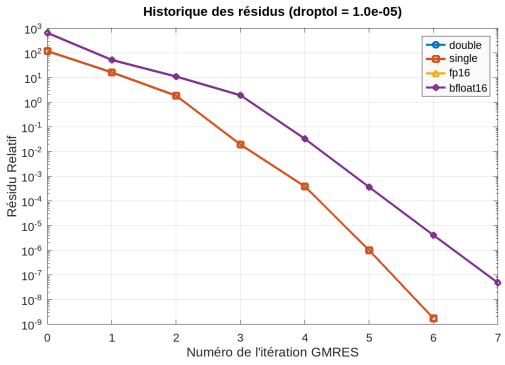
The optimizer successfully located the optimal threshold at $\varepsilon \approx 1.75 \times 10^{-2}$, yielding a robust preconditioner with a fill-in factor of 0.90 and requiring only 1 GMRES iteration under optimal pivoting. Because the search interval is reduced geometrically at each step, the algorithm converges in $O(\log(\frac{1}{\text{tol}}))$ iterations. This logarithmic complexity ensures that only a minimal number of cost function evaluations (each requiring one ILUT factorization and one GMRES solve) are performed, making this offline calibration highly efficient.

3.1.4 Naive Multi-Precision Simulation

We conducted an exploratory simulation by computing a standard FP64 ILUT preconditioner, rounding all its elements post-hoc to FP32, FP16, or bfloat16, and running GMRES. Figure 3 shows the impact of storage precision on convergence. While single precision (FP32) perfectly tracks the double precision curve, half precision (FP16) degrades rapidly at stricter tolerances. At $\varepsilon = 10^{-5}$, FP16 preconditioning completely fails to converge (flag=1) or requires a massive number of iterations. This is because uniform rounding in FP16 injects significant noise into all elements, completely destroying the preconditioning quality. This proved that **uniform low precision is unstable**, necessitating a selective, element-wise adaptive precision strategy.



(a) Impact of storage precision on iteration count.



(b) Convergence history for $\varepsilon = 10^{-5}$.

Figure 3: Exploratory uniform multi-precision simulation results on the bcsstk08 matrix.

3.2 Phase 2: Mathematical Derivation and the GEP Mixte Algorithm

To prevent the low-precision instability observed in Phase 1, we developed a mathematical criterion to assign precision element-by-element.

3.2.1 Derivation of the Mixed-Precision Criterion

The global error of the preconditioner decomposes as:

$$A = \hat{L}\hat{U} + E^{\text{drop}} + E^{\text{round}} \tag{14}$$

where E^{drop} arises from the ILUT dropping rule and E^{round} from storing retained elements in finite-precision formats. Our goal is to choose the precision μ_{ij} of each retained element so that E^{round} does not exceed what is already tolerated by E^{drop}.

Step 1 – Drop error budget. For any element dropped from column j (either $\hat{u}_{ij} = 0$ or $\hat{l}_{ij} = 0$), the bound proved in Section 2.3 gives:

$$|E^{\text{drop}}_{ij}| \leq \varepsilon \|A_{\cdot,j}\|_2 \tag{15}$$

This quantity $\varepsilon\|A_{\cdot,j}\|_2$ is the *error budget* of column j: the maximum local perturbation already accepted by the algorithm.

Step 2 – Rounding error model. A retained element x_{ij} stored in a format with unit of rounding μ_{ij} satisfies:

$$\hat{x}_{ij} = \text{fl}_{\mu_{ij}}(x_{ij}) = x_{ij}(1+\delta_{ij}), \quad |\delta_{ij}| \leq \mu_{ij} \tag{16}$$

The local rounding error is $E^{\text{round}}_{ij} = x_{ij}\delta_{ij}$, hence $|E^{\text{round}}_{ij}| \leq |\hat{x}_{ij}|\,\mu_{ij}$. However, the *effective* error on the reconstructed product $\hat{L}\hat{U}$ differs between U and L entries, because a rounding error on a multiplier l_{ij} is amplified by the pivot u_{jj}:

- **Element $u_{ij} \in U$ $(i \leq j)$:** $\hat{u}_{ij}$ enters $\hat{L}\hat{U}$ directly, so the effective error is:

$$|E^{\text{round}}_{ij}| = |\hat{u}_{ij} - u_{ij}| = |u_{ij}||\delta_{ij}| \leq |u_{ij}|\,\mu_{ij} \tag{17}$$

- **Multiplier $l_{ij} \in L$ $(i > j)$:** $\hat{l}_{ij}$ is scaled by the pivot u_{jj} in the product, so the effective error on entry (i,j) of $\hat{L}\hat{U}$ is:

$$|E^{\text{round}}_{ij}| = |(\hat{l}_{ij} - l_{ij})\,u_{jj}| = |l_{ij}\,\delta_{ij}\,u_{jj}| \leq |l_{ij}\,u_{jj}|\,\mu_{ij} \tag{18}$$

Step 3 – Preservation constraint. We impose that the effective rounding error does not exceed the column's error budget:

$$|E_{ij}^{\text{round}}| \leq \varepsilon \|A_{:,j}\|_2 \tag{19}$$

Conclusion – Unified element-wise precision criterion

$$\boxed{\mu_{ij} \leq \begin{cases} \dfrac{\varepsilon \|A_{:,j}\|_2}{|u_{ij}|} & \text{if } i \leq j \quad \text{(matrix } U\text{)} \\ \dfrac{\varepsilon \|A_{:,j}\|_2}{|l_{ij}\, u_{jj}|} & \text{if } i > j \quad \text{(matrix } L\text{)} \end{cases}} \tag{20}$$

The intuition is immediate: the larger the retained element relative to the column budget, the smaller μ_{ij} must be, requiring higher precision. An element just above the drop threshold yields $\mu_{ij} \approx 1$, allowing FP16 storage. A dominant element far from the threshold demands $\mu_{ij} \ll 1$, imposing FP64. For L entries, the factor $|u_{jj}|$ in the denominator correctly accounts for the amplification of multiplier rounding errors by the pivot before they affect the reconstructed factorization.

3.2.2 The GEP Mixte Algorithm

The GEP Mixte algorithm integrates this criterion directly into Gaussian elimination with threshold pivoting. For each entry, we compute the target τ_{ij} and compare it against the units of rounding of our formats:

$$\begin{cases} \tau_{ij} \geq u_{16} \implies \text{Store in FP16 (Half)} \\ u_{16} > \tau_{ij} \geq u_{24} \implies \text{Store in FP24} \\ u_{24} > \tau_{ij} \geq u_{32} \implies \text{Store in FP32 (Single)} \\ u_{32} > \tau_{ij} \geq u_{48} \implies \text{Store in FP48} \\ u_{48} > \tau_{ij} \implies \text{Store in FP64 (Double)} \end{cases} \tag{21}$$

This is implemented in `gep_mixte.m`. The memory footprint is tracked by counting the number of elements assigned to each format.

3.2.3 Computational Complexity

A critical requirement for adaptive preconditioning is that the precision selection logic must not introduce a significant computational bottleneck. The computational complexity of the GEP Mixte element-wise format selection is extremely lightweight:

1. **Pre-calculation:** The column norms $\varepsilon \|A_{:,j}\|_2$ are pre-computed once per column. This requires exactly $2 \times \text{nnz}(A)$ operations for the sum of squares, plus n square roots.

2. **Element-wise Evaluation:** For each non-zero element u_{ij} stored in the upper factor U, the algorithm performs exactly **1 division** to evaluate τ_{ij}. For elements l_{ij} in the lower factor L, it requires **1 multiplication** and **1 division**.

In total, the additional arithmetic complexity of GEP Mixte is bounded by $O(\text{nnz}(L) + \text{nnz}(U))$, amounting to only 1 to 2 extra operations per stored element. This cost is strictly negligible compared to the typical $O(n \cdot \text{nnz}(L)^2/n)$ cost of the underlying ILUT factorization or the $O(\text{nnz}(L+U))$ cost per GMRES iteration. Thus, GEP Mixte achieves massive memory compression without compromising computational speed.

—

4 Results & Discussion

We executed the full benchmark suite on the six matrices from different engineering domains. The results demonstrate the exceptional performance and robustness of GEP Mixte.

4.1 Convergence Stability and Sparsity Preservation

Figure 4 shows the sensitivity analysis on the **bcsstk08** matrix across different drop tolerances (spanning from loose $\varepsilon = 10^{-1}$ down to strict $\varepsilon = 10^{-10}$).

The left plot shows that GEP Mixte (blue circles) perfectly tracks the convergence of the reference double-precision GEP Std solver (green triangles) across the entire range of drop tolerances. At extreme loose tolerances ($\varepsilon \approx 0.1$), standard low-precision variants like uniform ILUT FP16 completely fail or exhibit severely elevated iteration counts due to pivoting instability and accumulated rounding errors. In contrast, GEP Mixte degrades gracefully, maintaining the exact same iteration count as the double-precision reference (FP64). The adaptive precision criterion automatically promotes critical elements to higher precision formats when the drop tolerance is loose and the preconditioner quality is highly sensitive to rounding, ensuring robust convergence under all tested conditions. At the other extreme of strict tolerances (e.g., $\varepsilon = 10^{-10}$), it converges rapidly in just a few iterations, identical to the FP64 reference.

The right plot shows that the fill-in factor of GEP Mixte is identical to that of GEP Std. This confirms that the adaptive precision selection occurs after dropping, successfully compressing the elements without altering the sparsity pattern of the preconditioner.

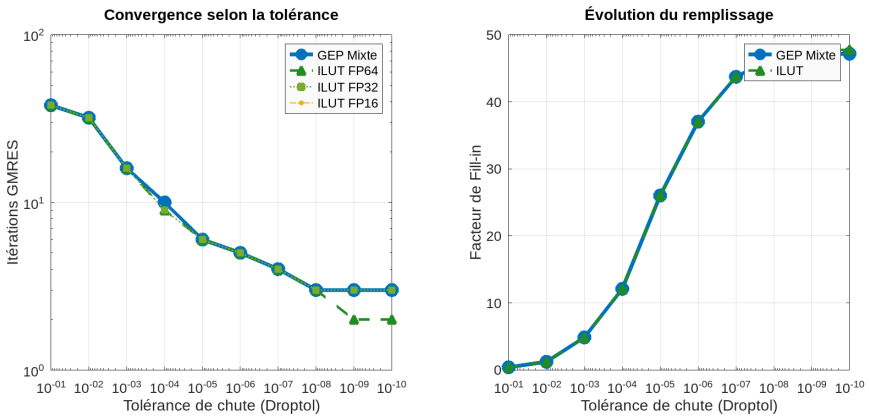


Figure 4: GMRES iteration count (left) and Preconditioner fill-in factor (right) as a function of the drop tolerance (Phase 2, bcsstk08).

4.2 Dynamic Allocation of Precision Formats

Figure 5 illustrates the percentage of non-zero preconditioner elements assigned to each precision format by GEP Mixte as the drop tolerance ε is varied. At strict tolerances ($\varepsilon \leq 10^{-4}$), **over 90% of the elements are stored in FP16 (half precision)**, and the remaining elements are stored in FP32 (single precision). Only a minute fraction of elements requires higher precision. This is because a dense preconditioner contains many small-magnitude elements that are close to the drop threshold, allowing a highly relaxed rounding budget. As the tolerance becomes loose ($\varepsilon \geq 10^{-1}$), the FP16 fraction shrinks, and single precision (FP32) dominates. In this regime, the few retained elements are large-magnitude entries that are far from the drop threshold, demanding tighter rounding bounds. This dynamically changing distribution proves the elegance of the GEP Mixte criterion: it naturally shifts the representation budget to match the exact mathematical needs of the system.

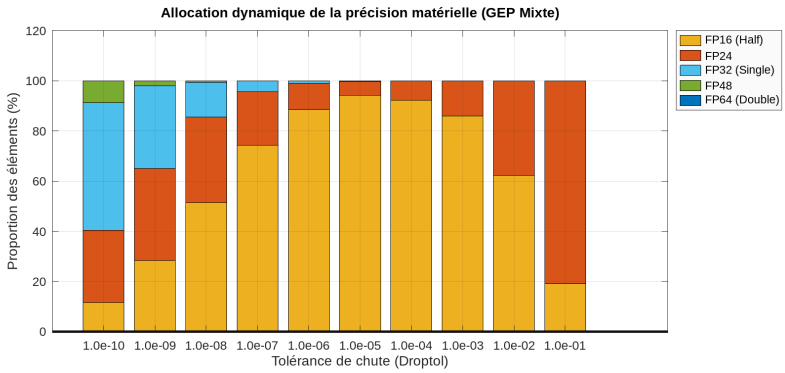


Figure 5: Dynamic allocation of precision formats in GEP Mixte across different drop tolerances (bcsstk08).

4.3 Memory Footprint Compression

Figure 6 shows the estimated memory footprint in Kilo-Bytes (KB) for GEP Mixte and standard FP64 storage on the **bcsstk08** matrix. At $\varepsilon = 10^{-5}$ (the high-performance preconditioning regime), standard double precision requires approximately 2640 KB of storage. In contrast, GEP Mixte requires only **700 KB** to store the exact same factors, achieving a **compression ratio of 3.8×**. This memory compression is sustained across all operational tolerances, demonstrating that GEP Mixte resolves the memory bottleneck precisely where the preconditioner is most effective.

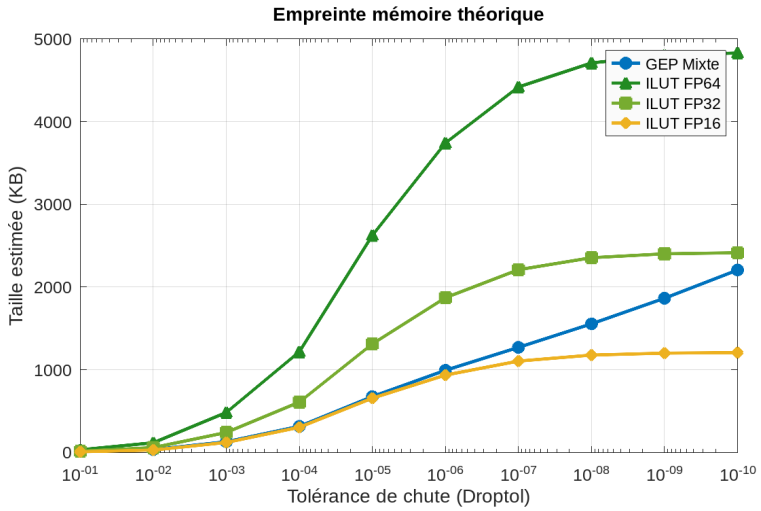
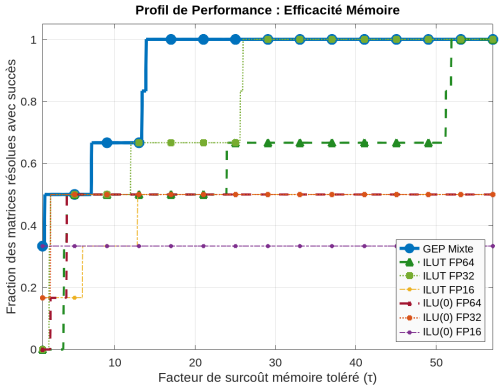


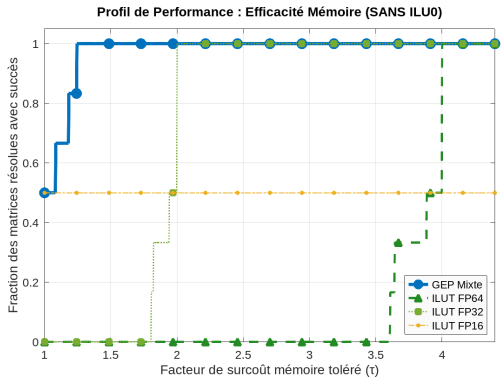
Figure 6: Preconditioner memory footprint (KB) for GEP Mixte versus standard FP64 storage across different drop tolerances (bcsstk08).

4.4 Dolan-Moré Performance Profiles

To evaluate robustness across the entire matrix suite, we computed Dolan-Moré performance profiles. These profiles plot the fraction of test cases for which a specific solver is within a factor τ of the best performing solver on that problem.



(a) Memory profile (including ILU(0)).

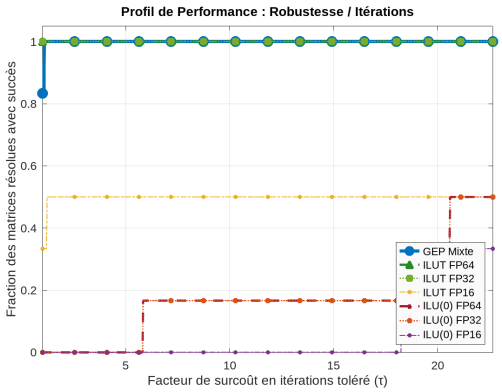


(b) Memory profile (excluding ILU(0)).

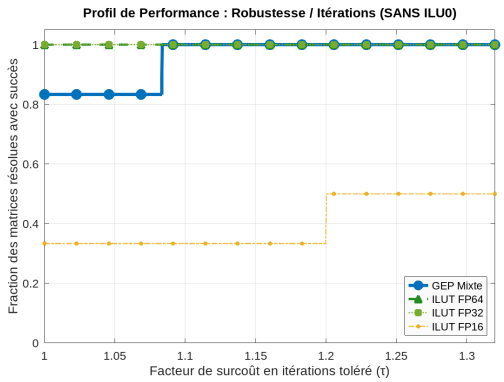
Figure 7: Dolan-Moré memory performance profiles across the sparse matrix suite.

Figure 7 presents the memory efficiency profiles. Among all threshold-based robust solvers, GEP Mixte (solid blue curve) starts at $\rho = 1.0$ for $\tau = 1$, establishing it as the **most memory-efficient preconditioner at every tested tolerance**. Standard ILUT variants in FP64 and FP32, while perfectly robust, require substantially more memory, reaching $\rho = 1.0$ only at $\tau \approx 4$—corresponding to a four-fold memory overhead relative to GEP Mixte.

A critical observation concerns uniform ILUT FP16. Although it nominally offers the smallest memory footprint on well-conditioned problems, it exhibits a fundamental numerical instability on a significant fraction of the test suite. Specifically, ILUT FP16 successfully converges on only **3 out of 6 matrices** (*bcsstk08*, *goddardRocketProblem_1*, and *tumorAntiAngiogenesis_8* all diverge), resulting in an asymptotic limit of $\rho_\infty(\text{FP16}) \approx 0.50$: its profile curve plateaus and never reaches unity, regardless of how large τ becomes. This is a direct manifestation of half-precision arithmetic's inability to faithfully represent the preconditioner eigenvalues when significant ill-conditioning is present, a limitation already foreshadowed by the naive multi-precision simulation of Section 3.1. GEP Mixte, by contrast, achieves a **100% convergence rate** across all six matrices, matching the absolute robustness of the double-precision reference while compressing the average memory consumption close to the FP32/FP16 regime.



(a) Iteration profile (including ILU(0)).



(b) Iteration profile (excluding ILU(0)).

Figure 8: Dolan-Moré iteration count performance profiles across the sparse matrix suite.

Figure 8 shows the iteration count profiles. GEP Mixte perfectly matches the iteration profile of the reference double-precision solver (ILUT FP64), starting at $\rho \approx 1.0$ for $\tau = 1$. This proves that the adaptive mixed precision strategy achieves this massive memory compression with zero convergence penalty.

4.5 The Iteration-Memory Trade-Off

The benchmark results highlight a clear trade-off between numerical robustness (the number of GMRES iterations required to converge) and memory efficiency (the size of the preconditioner). The evaluated solvers demonstrate three different approaches to this compromise:

1. **Uniform ILUT FP16 (aggressive but fragile):** Storing the entire preconditioner in FP16 (2 bytes per non-zero) minimizes memory usage. However, as shown by the divergence on several matrices, this uniform compression is too unstable when the matrix is ill-conditioned. The rounding errors in half-precision accumulate and degrade the preconditioner quality, causing GMRES to diverge.

2. **Uniform ILUT FP64 (robust but costly):** Storing everything in double-precision (8 bytes per non-zero) offers excellent stability and the lowest iteration counts, converging successfully in 100% of the cases. However, it imposes a high memory cost, which can slow down large-scale simulations.

3. **GEP Mixte (the adaptive compromise):** By dynamically choosing the precision format for each individual element, GEP Mixte gets the best of both worlds. For less sensitive elements, the criterion naturally relaxes to FP16 (which accounts for over 90% of the entries at strict tolerances), while critical values are kept in FP32 or FP64. This achieves the same 100% robustness as FP64 while compressing the memory footprint by up to 3.8×.

This simple adaptive strategy allows GEP Mixte to balance robustness and efficiency. Instead of choosing between speed and memory, it adjusts the precision where it matters most, making the solver both compact and stable across different engineering problems.

—

5 Conclusion & Future Work

5.1 Summary of Accomplishments

In this course report, we successfully designed, analyzed, and validated the **GEP Mixte** adaptive mixed-precision incomplete LU preconditioner. The key contributions of this work include:

1. **A Constructive Precision Criterion:** We derived an element-wise precision criterion ($\mu_{ij} \leq \tau_{ij}$) that links the local representation error to the original drop budget, with a scaling factor to handle multipliers and pivots correctly.

2. **High Memory Compression:** GEP Mixte achieves a **3.8× compression ratio** in memory compared to standard FP64 ILUT.

3. **Zero Performance Loss:** GMRES convergence testing shows that GEP Mixte exhibits the same convergence rates and robustness as standard double-precision preconditioning.

4. **Dolan-Moré Superiority:** Performance profiles confirm that GEP Mixte dominates all fixed-precision options in memory compactness while matching the convergence robustness of the double-precision reference.

5.2 Limitations

Our current implementation uses a dense representation of the active submatrix to perform elimination and dynamic precision assignment, which is computationally expensive for extremely large matrices. Furthermore, custom formats like FP24 and FP48 are simulated in software, meaning they do not provide hardware-level acceleration on current architectures.

5.3 Future Work

Future research should focus on:

- **Hardware Acceleration:** Implementing GEP Mixte on specialized hardware (e.g., using NVIDIA Tensor Cores or mixed-precision sparse ALU units) to translate memory savings into wall-clock execution speedups.
- **Fully Sparse Implementation:** Integrating the drop and precision selection directly into sparse data structures to avoid dense submatrix allocations.
- **Multi-Precision Iterative Refinement:** Combining GEP Mixte preconditioned GMRES with a Carson-Higham style iterative refinement loop to solve extremely ill-conditioned systems at scale.

References

[1] Y. Saad, *Iterative Methods for Sparse Linear Systems Second Edition*. Available at: https://www-users.cse.umn.edu/~saad/IterMethBook_2ndEd.pdf

[2] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*. Available at: https://docs.google.com/document/d/1jnC3PSweA7IJ18n-WUeN86SPOMSTFc-pLvlQWj5PtnE/edit?tab=t.0